

SYNCRO 5

ADDVERB

COBOT

Software Manual



Addverb's
Advanced
Robotics
Solutions ●

● material handling, assembly,
packaging, machine tending,
quality control, and even
dispensing and finishing tasks.

Contents

CHAPTER 1: UNDERSTANDING SOFTWARE	3
1.1 COBOT SOFTWARE INSTALLATION	3
1.2 USER SYSTEM SETUP AND COMPILATION	3
1.3 CONFIGURATIONS	3
1.3.1 <i>Payload</i>	3
1.3.2 <i>Safety type</i>	4
1.4 EXECUTION	5
1.5 CONTROLLERS OVERVIEW	5
1.6 AVAILABLE CONTROLLERS	6
1.6.1 <i>Recorder Controller</i>	6
1.6.2 <i>Free Drive Controller</i>	8
1.6.3 <i>Joint Jogging Controller</i>	8
1.6.4 <i>Cartesian Jogging Controller</i>	10
1.6.5 <i>Joint Impedance Controller</i>	11
1.6.6 <i>Cartesian Impedance Controller</i>	13
1.6.7 <i>Velocity Controller</i>	15
1.6.8 <i>PTP Joint Controller</i>	16
1.6.9 <i>PTP TCP Controller</i>	17
1.6.10 <i>Effort Controller</i>	19
1.6.11 <i>Gravity Compensation Effort Controller</i>	20
1.6.12 <i>Gripper Controller</i>	21
1.7 SHUTDOWN SERVICE AND ERROR RECOVERY	23
1.8 READ FT SENSOR DATA	24
1.9 READ END EFFECTOR DATA	24
1.10 TROUBLESHOOTING	25
1.10.1 <i>If the controllers are not working</i>	25
1.10.2 <i>If gripper is not working</i>	25
1.10.3 <i>Scripts on the Cobot not working</i>	25
1.10.4 <i>Unable to connect to the Cobot</i>	25

Chapter 1: Understanding Software

1.1 Cobot Software Installation

Detailed installation steps are provided in the *setup.md* file.

1.2 User System Setup and Compilation

It is recommended that the user's computer runs **Ubuntu 22.04 LTS**.

The cobot can be operated either:

- a) **Using SDK Directly**, by connecting directly to the cobot, or
- b) **Through ROS2**, by running the provided ROS 2 packages

To use ROS2, the user's system must be configured accordingly. The required setup and compilation instructions are provided in the *setup.md* file.

1.3 Configurations

The following cobot parameters can be configured as per requirement.

NOTE: The configuration step must be done before proceeding with the 14.4 Execution section on page 59

1.3.1 Payload

Payload defines the properties of the attachment at the end effector of the cobot.

To modify the payload parameters:

```
nano ~/cobot_ws/cobot_ros2/addverb_cobot_control/config/payload_config.yaml
```

Description of parameters:

- **payload_status:** Expects a boolean value (**1 or 0**). Use **1**, when you have an attachment at the end-effector, **0** otherwise. If this field is set to **0**, further parameters need not be set in the YAML
- **gripper_type:** Expects an integer between 0 to 5, indicating the make of the gripper
 - **0:** No Gripper
 - **1:** Dynamixel Gripper
 - **2:** Robotiq Gripper
 - **3:** DH Gripper
 - **4:** Suction Gripper
 - **5:** Feetech Gripper

- **ft_type:** Expects an integer between 0 to 2, indicating the make of the torque sensor
 - **0:** No Sensor
 - **1:** Robotiq
 - **2:** Robotous
- **ft_rot:** When the force-torque (FT) sensor is mounted on the robot's end effector, its local coordinate frame may not align with the end effector frame, causing measurement errors. To correct this, a rotation matrix representing the rotational difference between the sensor's frame and the robot's end effector frame is applied on the robot side to transform the sensor data accordingly. The user must generate this matrix based on the observed frame difference to ensure proper alignment and smooth robot operation.
- **mass:** Expects a **double** value, indicating the mass (Kg) of the attachment
- **comx, comy, comz:** Expect a **double** value, representing the Centre of Mass (m) of the cumulative system attached at the end-effector
- **lxx, lyy, lzz, lxy, lxz, lyz:** Expect a **double** value, representing the Moment of Inertia (kg.m^2) of the cumulative system attached at the end-effector
- **Safety type:** Expects an integer between (0-2). Details of the same are given below in section 14.3.2

1.3.2 Safety type

The Cobot supports **three safety levels**, which define how it responds to external forces and constraints:

- **Safety 0**
 - Baseline safety mode
 - The robot can be physically stopped by hand
 - Human operators can work in the robot's vicinity with minimal risk, as the system ensures compliance and safety stops cobot
- **Safety type 1**
 - Reduced safety compared to Mode 0
 - Human accommodation is not guaranteed
 - This mode allows faster or less restricted operation but should not be used in close human collaboration scenarios
- **Safety type 2**
 - **Least safe when used in conjunction with human** – designed primarily for error recovery
 - Position and joint limits are reset to higher values, bypassing normal safety constraints
 - **Not recommended for regular use** – should only be applied when explicitly required to recover from faults

1.4 Execution

Follow README.MD for a quick start. And for an in-depth understanding of the controllers, or examples refer to the sections below in [Available Controllers](#)

NOTE: Follow the (Execution) section given in the README.MD. Controllers will be available only if the controller manager is launched. It is necessary to launch it before using any commands from the following sections

1.5 Controllers Overview

- **Checking Active Controllers:**

To list all controllers and see which ones are active, run:

```
ros2 control list_controllers
```

- **Switching Controllers:** Refer README.MD

1.6 Available Controllers

1.6.1 Recorder Controller

The Recorder Controller enables recording and replaying of cobot joint trajectories.

- Record mode: Captures joint positions at a specified frequency and saves them in a file with a user-defined label at ROS side.
- Replay mode: Loads a recorded trajectory and replays it on the cobot for a user provided number of iterations
- By default, the controller switches between Free Drive mode which allows user to freely move the robot by hand and Replay mode:
 - When the user chooses to record, the cobot switches to Free Drive mode
 - When the user chooses to replay, the cobot switches to Replay mode automatically

EXECUTION:

Prerequisite:

- a) Ensure the controller manager is active, if not launch using `cobot_control.launch.py`
- b) Switch controller and activate `recorder_controller`. Ensure other controllers are deactivated.
- c) Recording Trajectories
The controller exposes a service:
 - `/recorder_controller/record_mode`
 - Type: `addverb_cobot_msgs/srv/Record`
 - **Fields:**
 - label (string) → name of recording file
 - rate (int) → sampling frequency in Hz
 - enable (bool) → start (true) or stop (false) recording
- d) When recording starts, the cobot switches to Free Drive mode, allowing the user to guide it manually.

a) Example: Start Recording:

Command:

```
ros2 service call /recorder_controller/record_mode addverb_cobot_msgs/srv/Record
"{enable: true,label: "test",rate: 5}"
```

b) Example: Stop Recording:

```
ros2 service call /recorder_controller/record_mode addverb_cobot_msgs/srv/Record
"{enable: false,label: "test",rate: 5}"
```

c) Replaying Trajectories:

The controller provides an action server:

/recorder_controller/replay_mode

Type: addverb_cobot_msgs/action/Replay

Goal:

- label (string) → recording to replay
- iterations (int) → number of times to repeat

Feedback:

- Current joint positions
- Iteration count

Result:

"SUCCESSFUL" or "FAILURE"

When replay starts, the robot automatically switches to Replay mode.

d) Example: Replay a Recorded Motion:

ros2 action send_goal /recorder_controller/replay_mode

addverb_cobot_msgs/action/Replay "{label: 'pick_motion', iterations: 2}"

LIMITS:

- Max recording rate: 30 Hz – Rate at which logging is done
- Position tolerance: 0.03 rad – Position tolerance limit for replay mode tracking
- Time tolerance: 0.01 s – Time tolerance limit for replay mode tracking

NOTE: If tolerances are exceeded during replay, the action may fail

PRECAUTIONS:

- Only one recording or replay can run at a time
- Starting a new recording automatically cancels replay, and vice versa
- Avoid recording while external controllers are active this can lead to corrupted trajectories

1.6.2 Free Drive Controller

This controller switches the cobot to free drive mode. In this mode, users can physically move the cobot freely without resistance. The controller does not require any commands from the terminal or scripts.

EXECUTION:

Step 1:

Controller manager must be active. If not, launch it using `cobot_control.launch.py`

Step 2:

Switch controller to “free_drive_controller”

1.6.3 Joint Jogging Controller

When running:

- Listen for **joint jogging velocity commands** on a dedicated ROS_2 topic
- Map commands to its hardware interfaces
- Continuously update joint velocities until stopped, replaced, or limited by safety checks

DEFAULT BEHAVIOUR:

- **On activation:**

All joint velocities set to 0.0

- **On receiving no commands:**

Velocities remain 0.0

- **On invalid commands:**

Robot logs a warning

INPUT VARIABLE:

- The parameter `commands` is a string[] that lists hardware command interface names, such as "joint1/velocity". An example value is {linear: ["joint1/velocity", "joint2/velocity", ...]}, representing interfaces for controlling joint velocities
- The parameter is the ROS topic `~/joint_jogging`, which uses the message type `addverb_cobot_msgs/msg/JointJoggingVelocity`. This topic serves as the main command input and includes a `jvel_scaling_factor[]` array that specifies velocity scaling for each joint. For example, `[1.0, 0.0, 0.0, 0.0, 0.0, 0.0]` commands the robot to jog joint1 forward

BEHAVIOUR AND PARAMETER EFFECTS EXAMPLES:

- **Changing `jvel_scaling_factor[0]` from 1.0 → 1.5**

Cobot will jog joint 1 faster in the positive direction

EXECUTION:**Prerequisite:**

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program**Command:**

```
ros2 run examples demo_joint_jogging
```

Note: This command jogs each joint in loop.

METHOD 2: Send manual input**Command: Send motion**

```
ros2 topic pub /joint_jogging_controller/joint_jogging/command \
addverb_cobot_msgs/msg/JointJoggingVelocity \
'{jvel_scaling_factor: [1.0, 0.0, 0.0, 0.0, 0.0, 0.0]}'
```

1.6.4 Cartesian Jogging Controller

When running:

- Cobot listens for geometry_msgs/msg/Twist commands on a dedicated ROS 2 topic
- Maps the **linear and angular components**
- Continuously updates velocities until stopped, replaced, or prevented by safety limits

DEFAULT BEHAVIOUR:

- **ON activation:**
All velocities set to 0.0
- **No incoming command**
End-Effector stays stationary
- **Invalid commands**
Ignored, warning logged

INPUT VARIABLES:

- The **commands parameter** is a **string[]** listing hardware command interface names for controlling linear velocities, e.g., {linear: ["linear_x_velocity", "linear_y_velocity", "linear_z_velocity"]}
- The ROS topic ~/cartesian_jogging/command uses geometry_msgs/msg/Twist messages to control robot movement, using only linear.x, linear.y, and linear.z for direction and velocity. For example, {linear: {x: 1.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}} moves the robot along the X-axis

BEHAVIOUR AND PARAMETER EFFECT EXAMPLES:

Change	Effect
Set linear.x = 1.0	Moves end-effector forward along +X axis
Set linear.z = -0.5	Moves end-effector down at half speed

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using cobot_control.launch.py

METHOD 1: Run the demo program

Command:

```
ros2 run examples cartesian_jogging
```

METHOD 2: Send manual input

Command 1: Send motion

Example: Send a simple +Y jog command:

```
ros2 topic pub /cartesian_jogging_controller/cartesian_jogging/command \
  geometry_msgs/msg/Twist \
  '{linear: {x: 0.0, y: 1.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}'
```

1.6.5 Joint Impedance Controller

Impedance control allows you to command both the target joint positions and the mechanical stiffness and damping of each joint, making the motion compliant or rigid depending on your needs.

When running, the controller will:

- Accepts a goal trajectory that contains joint positions and time to reach each point
- Apply stiffness and damping gains per joint
- Continuously track the trajectory until it completes, or violates constraints
- Validate all incoming data for safety and reject invalid command

DEFAULT BEHAVIOUR:

- **On activation:**
Stiffness and damping set to configured values
- **On invalid command:**
Command ignored
Stiffness and damping reset to defaults if invalid

INPUT VARIABLES:

- The **parameter commands** is a **string[]** that lists hardware command interface names for positions, such as ["joint1/position", "joint2/position", "joint3/position", ...], representing the interfaces used to control joint positions
- The **parameter stiffness** is a **double[]** representing per-joint stiffness gains. For example, [200.0, 200.0, 200.0, 200.0, 200.0, 200.0] sets the stiffness value for each joint
- The **parameter damping** is a **double[]** specifying per-joint damping gains. For example, [5.0, 5.0, 5.0, 5.0, 5.0, 5.0] sets the damping values for each joint.
- The parameter is the ROS action ~/follow_joint_trajectory of type control_msgs/action/FollowJointTrajectory serves as the main command input and includes trajectory.joint_names along with trajectory.points, which contain joint positions and timing information through positions[] and time_from_start.

BEHAVIOR AND PARAMETER EFFECT EXAMPLES:

- **Increasing stiffness[0] from 200.0 → 400.0**
Joint 1 becomes more rigid, resisting external forces
- **Providing an invalid stiffness vector length**
Falls back to default gains and logs a warning
- **Setting goal_time_tolerance too small** (Where **goal_time_tolerance** is the extra time after the segment end time allowed to reach the goal)
Note: Falls back to default tolerance (1.2 s) with a warning

EXECUTION

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program:

`ros2 run examples demo_joint_impedance`

NOTE: This above method defines a ROS 2 demo node that **sends a multiple joint trajectory point** to the `joint_impedance_controller`.

METHOD 2: Send manual input:

Param call to change the stiffness and damping:

`ros2 param set /joint_impedance_controller stiffness "[200.0, 200.0, 200.0, 200.0, 200.0, 200.0]"`

`ros2 param set /joint_impedance_controller damping "[5.0, 5.0, 5.0, 5.0, 5.0, 5.0]"`

Command: Send Motion

```
ros2 action send_goal /joint_impedance_controller/follow_joint_trajectory
control_msgs/action/FollowJointTrajectory "{
  trajectory: {
    joint_names: ['joint1', 'joint2', 'joint3', 'joint4', 'joint5', 'joint6'],
    points: [
      {
        positions: [0.1, -0.2, 0.3, -0.4, 0.5, -0.6],
        time_from_start: {sec: 10, nanosec: 0}
      }
    ]
  },
  goal_time_tolerance: {sec: 1, nanosec: 4}
}"
```

WARNINGS AND ERRORS:

Log Message	Meaning
Stiffness param invalid or missing. Using default	Incorrect stiffness array size
Damping param invalid or missing. Using default	Incorrect damping array size
Goal validation failed: joint_names is empty	Invalid goal
Joint Impedance data validation failed.	Impedance data stiffness and damping matrix are invalid
Incorrect time tolerance provided, using default value	Time tolerance is less than default time tolerance (1.2 seconds)

PRECAUTION

- **Parameter size checks:**
stiffness and damping arrays must each be length 6 (for 6 Cartesian DOF)
- **Trajectory constraints:**
 - `joint_names.size()` must match each point's `positions.size()`
 - The 'time_from_start' values must increase strictly with each point

EXAMPLE: Suppose the goal time for second point occurs before the goal time of first point then the trajectory is incorrect

- **Execution safety:** If validation fails mid-motion, impedance resets to defaults and error logged

1.6.6 Cartesian Impedance Controller

Unlike the Joint Impedance Controller which regulates stiffness and damping per joint, this controller regulates the cobot's end-effector behaviour in Cartesian space

With Cartesian impedance, you can command both:

- A desired end-effector trajectory (pose over time)
- The mechanical stiffness and damping in each Cartesian direction (X, Y, Z, and rotations), making the cobot compliant or rigid in task space

When running, the controller will:

- Accepts a goal trajectory that contains cartesian targets and time to reach each target
- Apply stiffness and damping gains in cartesian space
- Track the trajectory until completion, or constraint violation
- Validate all incoming data for safety

DEFAULT BEHAVIOUR:

- **On activation:**
Stiffness and damping set to configured values (or defaults)
- **On invalid command:**
Command ignored, stiffness and damping reset to defaults if invalid

INPUT VARIABLES:

- The **commands** parameter is a **string[]** that defines hardware interfaces for Cartesian position/orientation control, such as ["end_effector/pose"]
- The **stiffness** parameter is a **double[6] array** that defines Cartesian stiffness gains for the X, Y, Z translational axes and the Rx, Ry, Rz rotational axes. An example is [200.0, 200.0, 200.0, 20.0, 20.0, 20.0]
- The **damping** parameter is a **double[6] array** specifying Cartesian damping gains for the X, Y, Z translational axes and the Rx, Ry, Rz rotational axes. An example value is [5.0, 5.0, 5.0, 1.0, 1.0, 1.0]

- The **Mass_matrix parameter** is a **double[6]** array representing the diagonal mass terms used in Cartesian impedance control. An example is [1, 1, 1, 0.1, 0.1, 0.1]
- The **ft_force parameter** is a **double[6]** array that represents the measured or estimated wrench to be passed to the hardware.. An example is [0, 0, 0, 0, 0, 0]
- The **target_force parameter** is a **double[6]** array that defines the desired wrench to be tracked. An example value is [0, 0, 10, 0, 0, 0]

BEHAVIOR AND PARAMETER EFFECT EXAMPLES:

- **Increasing stiffness[0] from 200.0 → 500.0**
End-effector resists user interference more strongly along X-axis
- **Sending a trajectory with multiple Cartesian waypoints**
The controller streams through each pose with the given timing
- **Providing an invalid stiffness vector length**
Falls back to defaults and logs a warning
- **Setting goal_time_tolerance too small**
Reverts to default (1.2 s) and logs a warning

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program:

Command:

```
ros2 run examples demo_cartesian_impedance
```

METHOD 2: Send manual input:

Param call to change the stiffness and damping:

```
ros2 param set /cartesian_impedance_controller stiffness "[200.0, 200.0, 200.0, 200.0, 200.0, 200.0]"
```

```
ros2 param set /cartesian_impedance_controller damping "[5.0, 5.0, 5.0, 5.0, 5.0, 5.0]"
```

Command: Send Motion

```
ros2 action send_goal /cartesian_impedance_controller/follow_joint_trajectory
control_msgs/action/FollowJointTrajectory "{
  trajectory: {
    joint_names: ['joint1', 'joint2', 'joint3', 'joint4', 'joint5', 'joint6'],
    points: [
      {
        positions: [0.1, -0.2, 0.3, -0.4, 0.5, -0.6],
        time_from_start: {sec: 10, nanosec: 0}
      }
    ]
  },
  goal_time_tolerance: {sec: 1, nanosec: 4}
}"
```

1.6.7 Velocity Controller

- This controller allows user to directly command velocity for each joint of the cobot.
- Expects input array of 6x1, where each element in the array represents the velocity command for that particular joint. (Joints index starts from zero). Magnitude is in rad/s

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program

Command:

`ros2 run examples demo_velocity`

NOTE:

- The target = `[-0.01,0,0,0,0,0]` is sent and the cobot's base joint will keep moving at a velocity of 0.01 rad/s in clockwise direction until you stop the robot.

METHOD 2: Send manual input

Command:

```
ros2 topic pub /velocity_controller/commands std_msgs/msg/Float64MultiArray "{
  data: [0.1, 0, 0, 0, 0, 0]
}"
```

NOTE:

- `[0.1,0,0,0,0,0]` means publishing velocity of 0.1 rad/s to joint_1 , 0 rad/s to joint_2 and so on and so forth, If the user has sent any command, example: `[0.1,0,0,0,0,0]`, the robot will keep moving in that direction till the user explicitly stops the robot
- Stopping the robot can be done by modifying the data members of the array to `[0,0,0,0,0,0]`.
- With positive and negative values meaning motion in counterclockwise direction and clockwise direction respectively
- Default behaviour is to keep publishing zero velocities
- Any command which exceeds the limits will be rejected and warning published on the console

1.6.8 PTP Joint Controller

- This controller moves the cobot to one or more target positions within a specified time. Users can define position and time tolerances to allow for slight deviations and timing flexibility.
- Each target point must have only 6 values in the positions field, any value in velocity, acceleration and effort fields are ignored
- The controller rejects any new goal if a previous goal is already in execution
- You can constantly poll the joint_states as part of the feedback. Once the target position is achieved and target time lapsed, the controller updates the response of the request as detailed in the control_msgs/FollowJointTrajectory

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program

Command:

```
ros2 run examples demo_ptp_joint
```

NOTE:

- This demo script sends a predefined trajectory as goal to the server.

METHOD 2: Send manual input

Command: Send motion

```
ros2 action send_goal /ptp_joint_controller/follow_joint_trajectory
control_msgs/action/FollowJointTrajectory "{
  trajectory: {
    joint_names: ['joint1', 'joint2', 'joint3', 'joint4', 'joint5', 'joint6'],
    points: [
      {
        positions: [0.1, -0.2, 0.3, -0.4, 0.5, -0.6],
        time_from_start: {sec: 5, nanosec: 5}
      },
      {
        positions: [0.2, -0.1, 0.4, -0.3, 0.6, -0.5],
        time_from_start: {sec: 10, nanosec: 15}
      }
    ]
  },
  goal_time_tolerance: {sec: 1, nanosec: 20}
}"
```

NOTE: Notice that each subsequent target position has a goal time greater than previous positions.

1.6.9 PTP TCP Controller

- This controller allows user to give three or more target positions for end-effector of the cobot along with the target time to attain each target configuration
- Goal to this controller can be given by implementing an action client giving an action of type "FollowCartesianTrajectory" which is a custom message type

The components of this message type are explained:

- CartesianTrajectory is an array of CartesianTrajectoryPoint, where the Cartesian Trajectory Point has components:
 - **CartesianPoint**: target configuration
 - **time_from_start**: time to attain given configuration starting from the beginning
- CartesianPoint has components:
 - **geometry_msgs/Point**: target configuration x, y and z values
 - **geometry_msgs/Vector3**: target orientation roll, pitch and yaw values

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program

Command:

```
ros2 run examples demo_ptp_tcp
```

METHOD 2: Send manual input

Note: ptp_tcp needs a minimum of 3 points to execute a trajectory.

Command:

```
ros2 action send_goal /ptp_tcp_controller/follow_cartesian_trajectory
addverb_cobot_msgs/action/FollowCartesianTrajectory '{
  "trajectory": {
    "points": [
      {
        "point": {
          "position": {"x": 0.3, "y": 0.3, "z": 0.3},
          "orientation": {"x": 0.0, "y": 0.0, "z": 0.0}
        },
        "time_from_start": 3.0
      },
      {
        "point": {
          "position": {"x": 0.5, "y": 0.5, "z": 0.5},
          "orientation": {"x": 0.0, "y": 0.0, "z": 0.0}
        },
        "time_from_start": 7.0
      }
    ]
  }
}
```

```
    },  
    {  
      "point": {  
        "position": {"x": 1.0, "y": 1.0, "z": 1.0},  
        "orientation": {"x": 0.0, "y": 0.0, "z": 0.0}  
      },  
      "time_from_start": 10.0  
    }  
  ]  
}  
'
```

NOTE: Notice that each subsequent target point has a goal time greater than previous points.

PRECAUTIONS:

- Controller will reject a goal if target configuration has lesser than 3 points, a minimum of 3 points is required for multi-point target
- Ensure the coordinates are in workspace of the cobot

1.6.10 Effort Controller

- The Effort Controller expects joint torque commands (in Nm) as input and applies them directly to the robot joints, without adding gravity compensation torques. This provides full control to the user over the applied torques
- Once user has request to switch to effort controller, robot will first go to home configuration before switching to effort control mode
- The controller remains inactive until it receives a valid non-zero effort command from the user. When any joint receives a non-zero torque command, the controller transitions to active mode and begins processing effort commands

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

Launch Controller:

In `cobot_control_launch.py`, ensure the node for the controller spawner is included:

```
nodes = [..., effort_controller_spawner]
```

METHOD 1: Run the Demo Program

Command:

```
ros2 run example demo_effort
```

NOTE: Torque Limits:

Absolute Value: 60

Relative Value: 40

If these limits are exceeded, the robot will go in error.

METHOD 2: Send Manual Input

Command 1: Send Motion

```
ros2 topic pub /effort_controller/commands std_msgs/msg/Float64MultiArray "data:
[0.0, -22.0, 22.0, 0.0, 0.0, 0.0, 0.0]"
```

NOTE: This command activates the controller since a non-zero effort is commanded for one joint.

PRECAUTIONS

- The controller activates only when at least one joint receives a non-zero torque command
- This controller does not include gravity compensation; ensure your commanded torques account for gravity if necessary
- Apply gradual torque changes to prevent abrupt or hazardous robot behavior

1.6.11 Gravity Compensation Effort Controller

- The Effort Controller expects joint torque commands (in Nm) as input and applies them to the robot. It automatically adds gravity compensation torques to these external commands, making the total torque applied to each joint:

Total Torque = Gravity Compensation Torque + Commanded External Torque

- When zero torque is commanded or the controller is loaded without publishing commands, the robot behaves like freedrive mode

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

Launch Controller:

In `cobot_control_launch.py`, ensure the node for the controller spawner is included:

```
nodes = [...,gravity_comp_effort_controller]
```

using switch controller service switch to `gravity_comp_effort_controller`.

METHOD 1: Run the Demo Program

Command:

```
ros2 run example demo_gravity_comp_effort
```

METHOD 2: Send manual input

Command: Send Motion

```
ros2 topic pub /gravity_comp_effort_controller/commands std_msgs/Float64MultiArray  
"data: [0.0, 0.1, 0.0, 0.0, 0.0, 0.0]"
```

Precautions:

Avoid canceling gravity compensation: Sending external torques that directly oppose gravity compensation can reduce the total torque to near zero, causing the robot to fall under gravity. This may lead to unexpected motion or safety faults.

1.6.12 Gripper Controller

- The Gripper Controller provides a ROS 2 interface for commanding a cobot's gripper position
 - **Command mode:** Accepts user requests for gripper position and grasp force
 - **Feedback:** Returns success or failure of the requested operation
- By default, the controller stays idle until a valid service request is received. Users can command grasp force only for closing the gripper
- The Gripper Controller provides a validated and safe way to control the cobot gripper through ROS 2 services. It supports position and grasp force commands, ensures requests are within limits, and delivers feedback on execution success
- This makes it a reliable component for demonstrations, manipulation tasks, and automation workflows involving cobot grippers

EXECUTION:

Prerequisite:

Ensure the controller manager is active, if not launch using `cobot_control.launch.py`

METHOD 1: Run the demo program

Command:

```
ros2 run examples demo_gripper
```

NOTE:

The controller exposes a ROS 2 service:

- Service Name: `/gripper_controller/command`
- Type: `addverb_cobot_msgs/srv/Gripper`

Service Definition

Request:

- *float64 position*
- *float64 grasp_force*

Response:

- *bool success*
- *string message*

METHOD 2: Send manual input

Command 1: (Open Gripper):

```
ros2 service call /gripper_controller/command addverb_cobot_msgs/srv/Gripper
"{position: 1.0, grasp_force: 0.0}"
```

Response:

success: true

message: "success"

Command 2: (Close Gripper):

```
ros2 service call /gripper_controller/command addverb_cobot_msgs/srv/Gripper  
"{position: 0.0, grasp_force: 100.0}"
```

Response:

success: true

message: "success"

LIMITS

- Position values are normalized over various grippers and are in the range (0 to 1)
- Currently only position value (0 and 1) is supported
- Position value 0 (close) and 1 (open)
- Grasp force limits: 15 Kgf
- Commands are applied sequentially and may take 2 seconds to complete

PRECAUTIONS

- Invalid or unsafe requests are rejected with an error message
- Sending duplicate commands (e.g., close when already closed) does not move hardware
- Incorrect parameter mappings in payload.yaml can lead to wrong execution

1.7 Shutdown Service And Error Recovery

Service call for shutdown:

```
ros2 service call /cobot_services/shutdown_srv std_srvs/srv/Trigger {}
```

Shutdown Service (/cobot_services/shutdown_srv)

This service cleanly shuts down the robot but only succeeds if the cobot is **not in an error state**. If the cobot is in error, shutdown will fail. The server must already be launched on the cobot side, and the ROS 2 control manager must be running on the user's laptop for this service to work. The service is called a blocking call from the user's laptop Command line interface and returns success = true upon successful shutdown.

Service call for error recovery:

```
ros2 service call /cobot_services/error_recovery_srv std_srvs/srv/Trigger {}
```

Error Recovery Service (/cobot_services/error_recovery_srv)

This service resets the cobot from an error state and commands it to move back to the home position. It must be called before shutdown if the cobot is in error. Like shutdown, the server must be running on the cobot and the ROS 2 control manager on the user's laptop. This service is also called from the laptop's command line interface as a blocking call and returns success = true when recovery completes successfully.

Usage Flow:

If the cobot is in error, call the error recovery service first from your laptop's command line interface.

Then call the shutdown service from the laptop's command line interface power off the robot cleanly.

1.8 Read FT Sensor Data

If FT sensor has been provided along with the robot, user can read the data by subscribing to the topic `/ft_data`, published at 100 Hz.

To read the FT data through command line:
`ros2 topic echo /ft_data`

Details of the message type:

Data type of the published message is `geometry_msgs/msg/Wrench` where `geometry_msgs/msg/Wrench` consists of

- `geometry_msgs/Vector3` force: a 3x1 vector giving force values in X, Y and Z directions (in N)
- `geometry_msgs/Vector3` torque: a 3x1 vector giving torque values about X, Y and Z directions (in Nm)

1.9 Read End Effector Data

The user can read the end effector data by subscribing to the topic: `/ee_pos_data` published at 100 Hz.

- End effector vector (6X1) gives the position of the end effector in cartesian space

To read the End effector data through command line:
`ros2 topic echo /ee_pos_data`

Data type of published message is `addverb_cobot_msgs/msg/CartesianPoint` Where `addverb_cobot_msgs/msg/CartesianPoint` consists of:

- `geometry_msgs/Point` position: 3x1 vector giving position values in X, Y and Z directions (in m)
- `geometry_msgs/Vector3` orientation: 3x1 vector giving orientation values about X, Y and Z directions (in rad)

1.10 Troubleshooting

1.10.1 If the controllers are not working

- Check if multiple controllers are active
- Check if the desired controller is active. If not activate it
- Only the gripper_controller can run alongside another controller
- If multiple controllers are active, **deactivate all** and **reactivate** only the required one

1.10.2 If gripper is not working

- Verify that **payload_status** is set to 1
- Ensure the **gripper_type** parameter is correct
- Confirm that the gripper is **powered** and **properly connected** to the control box

1.10.3 Scripts on the Cobot not working

If the CLI halts after displaying specific messages:

- **"Port Initialized"**: Check if the E-Stop is pressed
- **"heard"**: The script may not be connecting to the controller manager
 - Possible cause: multiple instances of server running
 - Solution: kill duplicate processes or restart the cobot
- **"Setting relay on"** → Press the reset button on the PCB inside the control box, then retry running the script

1.10.4 Unable to connect to the Cobot

- Ensure the user's laptop IP address is static
- Verify that it matches the required network configuration for the cobot

ADDVERB



Plot No. 5, Sector 156, Noida, Uttar Pradesh 201301

+91 9871590101

automate@addverb.com

addverb.ai

